

Experiment No.12

1. **Title:** Traffic Singnal Simulation using Swing and AWT Graphics.

2. **Problem Statement:**

Write a Java program using Swing for GUI and AWT event handling that simulates a traffic light system. The interface should contain three radio buttons labeled Red, Yellow, and Green. Selecting any radio button should turn on the corresponding light in the traffic signal. Only one light may be on at a time, and when the application starts, no light should be illuminated. The program must visually display the traffic lights and update the display based on the user's selection.

3. **Objectives:**

- To design a graphical user interface (GUI) using Java Swing components.
- To implement event handling using the ActionListener interface.
- To use radio buttons and ButtonGroup for mutually exclusive selection.
- To simulate a traffic light system with dynamic visual updates based on user actions.

4. **Outcomes:**

After completing this program, students will be able to:

- Develop interactive GUI applications using Java Swing.
- Handle user-generated events using AWT event handling mechanisms.
- Apply radio button controls to manage single-option selection.
- Create and update graphical components dynamically using Java Graphics.

5. **Software/Hardware/Other Requirements:**

- Any CPU with Pentium Processor / similar, 256 MB RAM/ more, 1GB HDD / more.
- Operating System – ubuntu/Fedora 64bit OS
- Software: JDK 17 or later, Code Editor

6. **Theory:**

A. Graphics Class

- The **Graphics** class is part of the **java.awt** package. It provides methods for drawing shapes, text, images, and colors on GUI components such as JPanel, JFrame, and Canvas.
- Think of the Graphics object as a drawing tool or virtual pen that allows a program to draw on a component's surface.

import java.awt.Graphics;

- **Why Graphics Class is Required?**

- In a normal GUI, components like buttons and text fields are displayed automatically. However, when we need to draw: Circles, Rectangles, Lines, Polygons, Text, Images

- **Let's look at some of the methods available in the Graphics class:**

- 1) **void drawLine(int startX, startY, endX, endY)** – Used to draw a line between two points.
- 2) **void drawRect(int startX, startY, int width, int height)** – Used to draw a rectangle starting from the top left corner with the given width and height. The coordinates of the top left corner of the rectangle are startX and startY.
- 3) **void fillRect(int startX, startY, int width, int height)** – Used to draw a solid (colored) rectangle with the given parameters.
- 4) **void drawRoundRect(int startX, startY, int width, int height, int xDiam, int yDiam)** – Used to draw a rounded rectangle whose x-diameter and y-diameter of the corners is given by xDiam and yDiam respectively.
- 5) **void fillRoundRect(int startX, startY, int width, int height, int xDiam, int yDiam)** – Used to draw a solid (colored) rounded rectangle whose x-diameter and y-diameter of the corners is given by xDiam and yDiam respectively.
- 6) **void drawOval(int startX, startY, int width, int height)** – Used to draw an ellipse inside an imaginary rectangle whose dimensions are specified by the given parameters. We can get a circle by giving the same value for both width and height.

- 7) **void fillOval(int startX, int startY, int width, int height)** – Used to draw a solid (colored) ellipse inside an imaginary rectangle whose dimensions are specified by the given parameters. We can get a circle by giving the same value for both width and height.
- 8) **void drawArc(int startX, int startY, int width, int height, int startAngle, int sweepAngle)** – Used to draw an arc inside an imaginary rectangle. The start angle and the sweep angle are specified by using startAngle and sweepAngle respectively.
- 9) **void fillArc(int startX, int startY, int width, int height, int startAngle, int sweepAngle)** – Used to draw a solid (colored) arc inside an imaginary rectangle. The start angle and the sweep angle are specified by using startAngle and sweepAngle respectively.
- 10) **void drawPolygon(int x[], int y[], int numPoints)** – Used to draw a polygon whose x coordinates and y coordinates of the points are specified using the x array and y array respectively. Number of points are specified using numPoints.
- 11) **void fillPolygon(int x[], int y[], int numPoints)** – Used to draw a solid (colored) polygon whose x coordinates and y coordinates of the points are specified using the x array and y array respectively. Number of points are specified using numPoints.

B. Color Class

- To use colors in our GUI applications, AWT provides the Color class. We can create a Color object by using any of the following constructors:
 - Color(int red, int green, int blue)
 - Color(int rgbValue)
 - Color(float red, float green, float blue)
- We can get values of red component, green component or blue component of the color by using the following methods:
 - int getRed()
 - int getGreen()
 - int getBlue()
- We can apply or get the color of the graphic object by using the following methods:
 - void setColor(Color newColor)
 - Color getColor()

- **Example:**

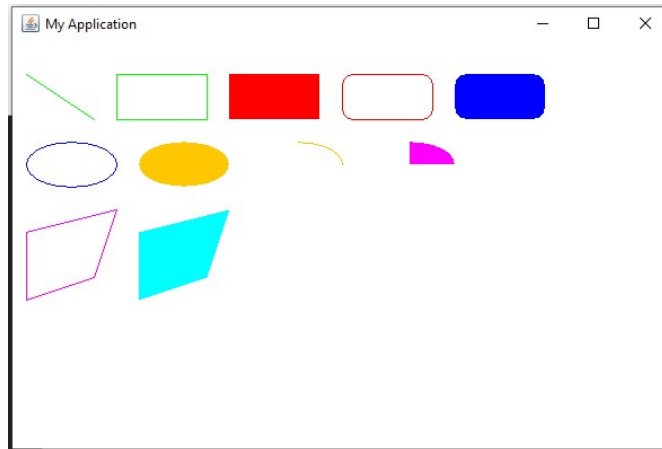
```
import java.awt.*;
import java.awt.event.*;
public class MyFrame extends Frame{
    MyFrame(){
        setSize(600, 400);
        setTitle("My Application"); setVisible(true);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we){ System.exit(0); } });
    }
    public void paint(Graphics g){
        g.drawLine(20, 60, 80, 100);
        g.drawRect(100, 60, 80, 40);
        g.fillRect(200, 60, 80, 40);
        g.drawRoundRect(300, 60, 80, 40, 20, 20);
        g.fillRoundRect(400, 60, 80, 40, 20, 20);
        g.drawOval(20, 120, 80, 40);
        g.fillOval(120, 120, 80, 40);
        g.drawArc(220, 120, 80, 40, 90, -90);
        g.fillArc(320, 120, 80, 40, 90, -90);
        int[] x = {20, 100, 80, 20};
        int[] y = {200, 180, 240, 260};
        g.drawPolygon(x, y, 4);
        int[] fillx = {120, 200, 180, 120};
        int[] filly = {200, 180, 240, 260};
        g.fillPolygon(fillx, filly, 4);
    }
}
```

```

public static void main(String[] args) {
    MyFrame mf = new MyFrame();
}

```

Output:



C. **paintComponent() Method**

- The paintComponent() is a method of the JPanel class used for custom drawing.
- Whenever Swing needs to display or redraw a component, this method is called automatically.

- **Syntax:**

```

public void paintComponent(Graphics g){
    super.paintComponent(g);
    // Drawing code
}

```

- **Why Override paintComponent()?**

- The default panel only shows its background. To draw: Traffic signals, Shapes, Graphs, Animations, Games, hence we override this method.

Example:

```

class MyPanel extends JPanel {
    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.drawRect(50,50,100,100);
    }
}

```

- **Relationship Between repaint() and paintComponent()**

repaint(): Requests the component to redraw itself.

```
repaint();
```

- **Internal Working**

```
repaint()
```

↓

```
paintComponent()
```

↓

Graphics Draw Again

- **Difference Between paint() and paintComponent()**

paint()	paintComponent()
Part of AWT	Part of Swing
Used for top-level containers	Used for components like JPanel
More complex	Easier and recommended
May paint children too	Focuses only on component drawing
Less commonly used in Swing	Most commonly used for custom graphics

- For Swing applications, always prefer **paintComponent()**.

D. JPanel

- JPanel is a container used to organize GUI components and perform custom drawing.
- **Advantages:**
 - Supports graphics rendering.
 - Can contain other Swing components.
 - Helps separate GUI layout from drawing logic.
- **Role in Program:** Used to draw the traffic signal body and lights.

E. Radio Buttons (JRadioButton)

- A radio button allows the user to select **one option from a set of choices**.
- **Properties:**
 - Represents mutually exclusive options.
 - Generates action events when selected.
- **Role in Program:**
 - Red Radio Button → Turns ON red light.
 - Yellow Radio Button → Turns ON yellow light.
 - Green Radio Button → Turns ON green light.

F. ButtonGroup

- A ButtonGroup groups multiple radio buttons so that only one can be selected at any given time.
- **Working:**

```
ButtonGroup group = new ButtonGroup();
group.add(redBtn);
group.add(yellowBtn);
group.add(greenBtn);
```
- **Importance:** Prevents multiple lights from being ON simultaneously, mimicking a real traffic signal.

G. Short Description of the program:

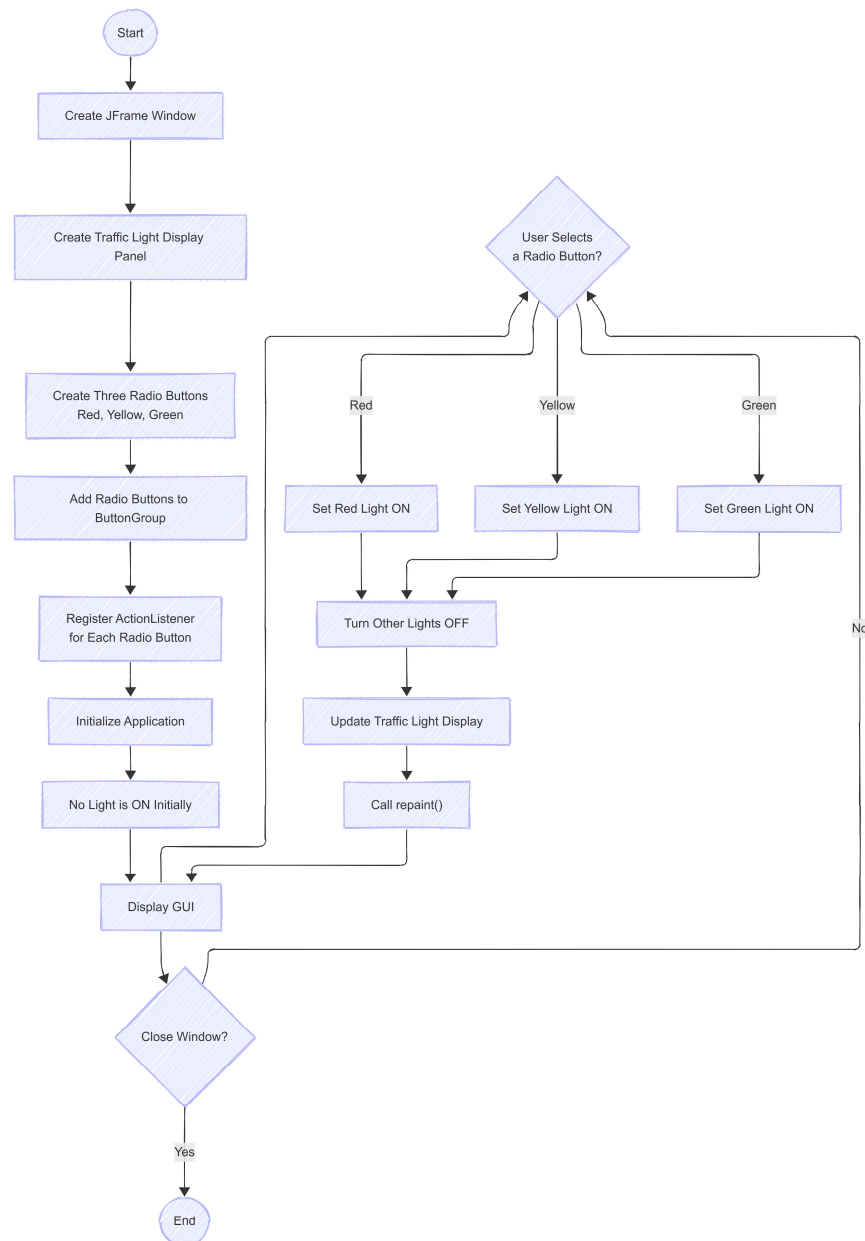
- Design and develop a **Traffic Light Simulation System** using **Java Swing for Graphical User Interface (GUI)** development and **AWT event handling** for user interaction. The application should display a graphical representation of a traffic signal containing three lights: **Red, Yellow, and Green**. The user interface must include three radio buttons labeled **Red, Yellow, and Green**, allowing the user to control the state of the traffic signal.
- When the application starts, all three lights should remain **OFF** (displayed in gray color). Upon selecting a radio button, the corresponding light should illuminate with its respective color while the remaining lights remain turned off. The program should ensure that only one radio button and one traffic light can be active at a time by using a **ButtonGroup**. The application must respond to user actions through the **ActionListener** interface and dynamically update the graphical display using the **Graphics** class and the **paintComponent()** method.
- The project aims to demonstrate the concepts of **GUI design, event-driven programming, custom graphics rendering, radio button controls, and dynamic screen updating** in Java. The completed application should provide an interactive and realistic simulation of a traffic light control system while helping students understand the practical implementation of Swing components and AWT event handling mechanisms.

7. Algorithm:

1. **Start** the program.
2. Create a JFrame to display the traffic light application.
3. Create three radio buttons: Red, Yellow, Green
4. Add all radio buttons to a ButtonGroup so that only one button can be selected at a time.
5. Initially, do not select any radio button and keep all traffic lights OFF.
6. Create a custom JPanel to draw the traffic signal.
7. Override the **paintComponent(Graphics g)** method to draw: Traffic light housing, Red circle, Yellow circle, Green circle.

8. Register an ActionListener for all three radio buttons.
9. Display the GUI window.
10. When the user selects a radio button: Identify the selected color., Turn ON the corresponding traffic light., Keep the other two lights OFF.
11. Call the repaint() method to refresh the traffic light display.
12. The paintComponent() method redraws the traffic signal with the selected light illuminated.
13. Continue accepting user selections until the application window is closed.
14. Stop the program

8. Flowchart



9. Implementation:

Java Program for implementation for Traffic Signal Simulation	
<pre>import javax.swing.*; import java.awt.*; import java.awt.event.*;</pre>	

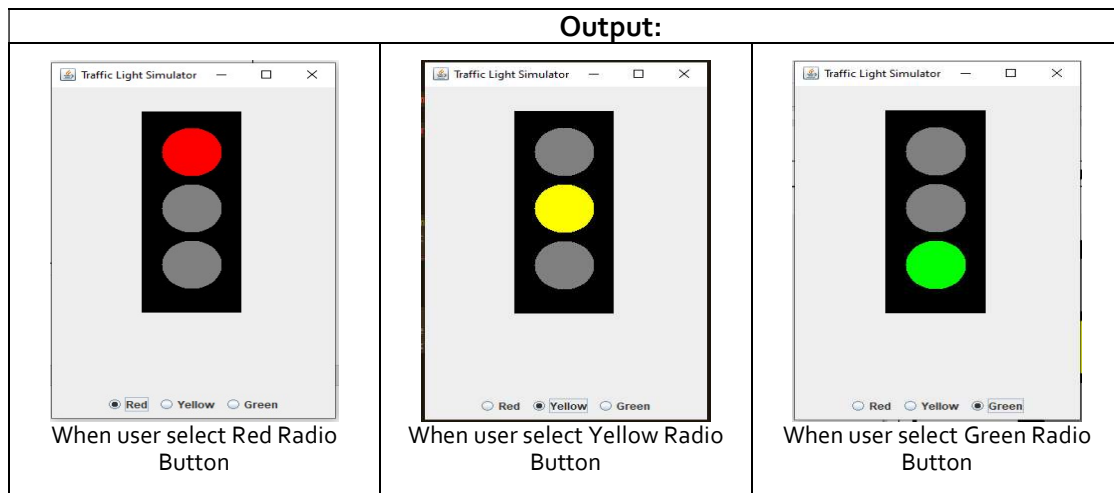
```

public class TrafficLightSimulator extends JFrame implements ActionListener {
    private JRadioButton redButton, yellowButton, greenButton;
    private ButtonGroup group;
    private TrafficPanel trafficPanel;

    public TrafficLightSimulator() {
        setTitle("Traffic Light Simulator");
        setSize(300, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());
        // Radio Buttons
        redButton = new JRadioButton("Red");
        yellowButton = new JRadioButton("Yellow");
        greenButton = new JRadioButton("Green");
        group = new ButtonGroup();
        group.add(redButton);
        group.add(yellowButton);
        group.add(greenButton);
        JPanel buttonPanel = new JPanel();
        buttonPanel.add(redButton);
        buttonPanel.add(yellowButton);
        buttonPanel.add(greenButton);
        redButton.addActionListener(this);
        yellowButton.addActionListener(this);
        greenButton.addActionListener(this);
        trafficPanel = new TrafficPanel();
        add(buttonPanel, BorderLayout.SOUTH);
        add(trafficPanel, BorderLayout.CENTER);
        setVisible(true);
    }
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == redButton)
            trafficPanel.setSignal("RED");
        else if (e.getSource() == yellowButton)
            trafficPanel.setSignal("YELLOW");
        else if (e.getSource() == greenButton)
            trafficPanel.setSignal("GREEN");
    }
    public static void main(String[] args) {
        new TrafficLightSimulator();
    }
}

class TrafficPanel extends JPanel {
    private String signal = "";
    public void setSignal(String signal) {
        this.signal = signal;
        repaint();
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        // Signal Box
        g.setColor(Color.BLACK);
        g.fillRect(90, 30, 100, 250);
        // Red Light
        if (signal.equals("RED"))
            g.setColor(Color.RED);
        else
            g.setColor(Color.GRAY);
        g.fillOval(110, 50, 60, 60);
        // Yellow Light
        if (signal.equals("YELLOW"))
            g.setColor(Color.YELLOW);
        else
            g.setColor(Color.GRAY);
        g.fillOval(110, 120, 60, 60);
        // Green Light
        if (signal.equals("GREEN"))
            g.setColor(Color.GREEN);
        else
            g.setColor(Color.GRAY);
        g.fillOval(110, 190, 60, 60);
    }
}

```



10. Conclusion:

Hence, we have successfully studied and implemented the Traffic Light Simulation using Java Swing and AWT Event Handling. The program demonstrates the use of GUI components, event handling, radio buttons, graphics drawing, and the paintComponent() method to create an interactive traffic signal where the selected light is illuminated while the others remain off.

11. Review Questions & Exercises:

a. Fill in the Blanks

Q. No.	Question	BT Level	CO	PO & PSO
i	The _____ class is used to group radio buttons so that only one can be selected at a time.	BT ₁	CO ₃	PO ₁ , PSO ₁
ii	The method _____ is used to request the component to redraw itself.	BT ₁	CO ₃	PO ₁ , PSO ₁
iii	Statement _____ is used to clear previous drawing before drawing new graphics.	BT ₁	CO ₃	PO ₁ , PSO ₁

b. Answer the following:

Q. No.	Question	BT Level	CO	PO & PSO
i	Explain the importance of event-driven programming in GUI applications.	BT ₁	CO ₃	PO ₁ , PSO ₁
ii	How would you modify the program to simulate a real traffic controller with automatic signal changes?	BT ₃	CO ₃	PO ₁ , PO ₂ , PSO ₁
iii	What is a JPanel and why is it used?	BT ₂	CO ₃	PO ₁ , PSO ₁

c. Programming Assignments:

Que. No.	Question	BT Level	CO	PO & PSO
i	Animated Fan Control System: Develop an Animated Fan Control System using Java Swing, AWT event handling, and Graphics programming. The application should display a graphical fan with rotating blades and provide an interactive user interface to control its operation. The GUI must include Start, Stop, and Reverse Rotation buttons to manage the fan's movement, along with Low, Medium, and High speed radio buttons for selecting predefined speed levels. A JSlider should be provided for fine-grained speed adjustment during runtime. The fan blades should rotate smoothly using a Swing Timer, and any changes made through buttons, radio buttons, or the slider should immediately affect the fan's speed and direction. The application should dynamically update the display using the Graphics class and the paintComponent() method, demonstrating concepts such as GUI design, event-driven programming, Timer-based animation, graphics rendering, and user interaction in Java	BT ₆	CO ₃	PO _{1,3, 5} , PSO ₁

Que. No.	Question	BT Level	CO	PO & PSO
ii	Elevator (Lift) Simulation System: Develop an Elevator (Lift) Simulation System using Java Swing, AWT Graphics, and Timer-based animation . The application should display a multi-floor building with an animated elevator that moves smoothly between floors. The user can select any floor (1 to 5) using floor selection buttons, and the elevator should automatically travel to the requested floor while displaying the current floor number and movement direction (Up/Down). The GUI should also include Open Door, Close Door, Emergency Stop, and Reset buttons to control the elevator's operation. When the elevator reaches the selected floor, a door-opening animation should be displayed, followed by a door-closing animation before accepting the next request. The Emergency Stop button should immediately halt the elevator, while the Reset button should return it to the ground floor. The application should dynamically update the display using the Graphics class, paintComponent() method, and Swing Timer , demonstrating concepts such as GUI design, event-driven programming, graphics rendering, animation, timers, buttons, labels, and interactive user control in Java.	BT6	CO3	PO _{1,2,3,5} , PSO ₁

12. References:

- Java 6 Programming, Black Book, Dreamtech
- Paul Deitel and Harvey Detail, Java: How to Program, Pearson's Publication, 9th Edition
- Herbert Schildt, (2021). Java: The complete reference, 13th edition. McGraw-Hill Education.

Performance Indicators	
Process related (6 Marks)	
1	Logic formation (3)
2	Debugging ability (2)
3	Follow ethical practices (1)
Product related (4 Marks)	
4	Expected output (1)
5	Timely Submission (2)
6	Answer to sample questions (1)

Marks Obtained			Dated signature of Teacher
Process Related (6)	Product Related (4)	Total (10)	